شرح سطر بسطر ١١ — شاشة 27: إدخال الرمز السري و«نسيت الرمز؟»

مشروع رِزمة · Flutter · كل سطر مع سبب كتابته بهذه الطريقة

حين يختار الطالب ملفًّا عليه رمز سري تظهر هذه الشاشة. هي قفل خفيف بين الإخوة وليست حماية أمنية قوية. تعتمد على `ProfileRepository.verifyPin` الذي يقرر: ناجح / خاطئ / مقفل (بعد 5 محاولات خاطئة لمدة 30 ثانية).

وإن نسي الطالب رمزه: «نسيت الرمز؟» تفتح ورقة تطلب رمز الطالب (`RZ-XXXX`)؛ إن صحّ يختار الطالب تعيين رمز جديد أو إلغاء الرمز، ثم يدخل مباشرة.

العدّ التنازلي المعروض (`_lockedSeconds`) محلي للعرض فقط؛ المرجع الحقيقي للقفل هو `PinLockout` داخل المستودع.

lib/features/profiles/pin_entry_screen.dart — شاشة إدخال الرمز

lib/features/profiles/pin_entry_screen.dart

السطر 1

```
1  import 'dart:async';
```

ماذا يفعل؟ استيراد `dart:async`.

لماذا بهذه الطريقة؟ من أجل `Timer` الخاص بالعدّ التنازلي.

الأسطر 3–5

```
3  import 'package:flutter/material.dart';
4  import 'package:flutter_riverpod/flutter_riverpod.dart';
5  import 'package:go_router/go_router.dart';
```

ماذا يفعل؟ استيراد Material وRiverpod وgo_router.

لماذا بهذه الطريقة؟ الشاشة تقرأ المزوّدات وتتنقّل.

الأسطر 7–19

```
import '../../app/routes.dart';
import '../../core/strings/ar.dart';
import '../../core/theme/app_colors.dart';
import '../../core/theme/app_text_styles.dart';
import '../../core/theme/profile_palette.dart';
import '../../core/utils/pin_hasher.dart';
import '../../core/widgets/mascot.dart';
import '../../core/widgets/pin_pad.dart';
import '../../core/widgets/screen_top_bar.dart';
import 'active_profile_provider.dart';
import 'forgot_pin_sheet.dart';
import 'profile_providers.dart';
import 'profile_repository.dart';
```

ماذا يفعل؟ استيراد المسارات والنصوص والألوان والأنماط وألوان الشخصية وأداة الرمز السري والشخصية ولوحة الرمز والشريط العلوي ومزوّدات الملفات وورقة «نسيت الرمز» والمستودع.

لماذا بهذه الطريقة؟ `profile_repository.dart` لنوع `PinCheck`.

الأسطر 21–25

```
/// إدخال الرمز السري لملف محمي — الشاشة 27.
class PinEntryScreen extends ConsumerStatefulWidget {
  const PinEntryScreen({super.key, required this.profileId});

  final String profileId;
```

ماذا يفعل؟ تعليق وصنف `PinEntryScreen` ومعاملها `profileId`.

لماذا بهذه الطريقة؟ تُفتح لملف بعينه.

الأسطر 27–29

```
  @override
  ConsumerState<PinEntryScreen> createState() => _PinEntryScreenState();
}
```

ماذا يفعل؟ `createState`.

لماذا بهذه الطريقة؟ للحالة المحلية.

السطر 31

```
class _PinEntryScreenState extends ConsumerState<PinEntryScreen> {
```

ماذا يفعل؟ صنف الحالة.

لماذا بهذه الطريقة؟ بحالة وبـ `ref`.

الأسطر 32–36

```
32    String _entry = '';
33    bool _wrong = false;
34    bool _busy = false;
35    int _lockedSeconds = 0;
36    Timer? _ticker;
```

ماذا يفعل؟ الحقول: الأرقام المُدخلة، وعلامة الخطأ، وعلامة الانشغال، والثواني المتبقية للقفل، والمؤقّت.

لماذا بهذه الطريقة؟ `_busy` يمنع الإدخال المزدوج أثناء التحقق.

السطر 38

```
38    bool get _isLocked => _lockedSeconds > 0;
```

ماذا يفعل؟ `_isLocked`: هل هناك ثوانٍ متبقية؟

لماذا بهذه الطريقة؟ خاصية مشتقة لا حالة مكرّرة.

الأسطر 40–44

```
40    @override
41    void dispose() {
42      _ticker?.cancel();
43      super.dispose();
44    }
```

ماذا يفعل؟ `dispose`: نلغي المؤقّت.

لماذا بهذه الطريقة؟ منع استدعاء `setState` بعد إغلاق الشاشة.

الأسطر 46–47

```
46    void _onDigit(String digit) {
47      if (_busy || _isLocked || _entry.length >= PinHasher.pinLength) return;
```

ماذا يفعل؟ `_onDigit`: نتجاهل الإدخال إن كان `_busy` أو مقفلًا أو اكتمل الطول.

لماذا بهذه الطريقة؟ أثناء القفل أو التحقق لا نقبل أرقامًا.

الأسطر 48–51

```
48      setState(() {
49        _wrong = false;
50        _entry += digit;
51      });
```

ماذا يفعل؟ نمسح علامة الخطأ ونضيف الرقم.

لماذا بهذه الطريقة؟ يبدأ الطالب محاولة جديدة.

الأسطر 53–52

```
52      if (_entry.length == PinHasher.pinLength) _verify();
53    }
```

ماذا يفعل؟ عند اكتمال أربعة أرقام نستدعي `_verify`.

لماذا بهذه الطريقة؟ لا زر «تأكيد»: التحقق تلقائي.

الأسطر 61–55

```
55    void _onDelete() {
56      if (_busy || _entry.isEmpty) return;
57      setState(() {
58        _wrong = false;
59        _entry = _entry.substring(0, _entry.length - 1);
60      });
61    }
```

ماذا يفعل؟ `_onDelete`: يحذف آخر رقم ويمسح علامة الخطأ.

لماذا بهذه الطريقة؟ إن كان `_busy` أو فارغًا لا يفعل شيئًا.

الأسطر 66–63

```
63    Future<void> _verify() async {
64      setState(() => _busy = true);
65      final repo = ref.read(profileRepositoryProvider);
66      final result = await repo.verifyPin(widget.profileId, _entry);
```

ماذا يفعل؟ `_verify`: نضع `_busy` ونستدعي `verifyPin` من المستودع.

لماذا بهذه الطريقة؟ المستودع يقرر النتيجة ويدير القفل.

السطر 67

```
67      if (!mounted) return;
```

ماذا يفعل؟ نخرج إن أُغلقت الشاشة أثناء الانتظار.

لماذا بهذه الطريقة؟ حماية `mounted`.

الأسطر 71–69

```
69      switch (result.check) {
70        case PinCheck.ok:
71          await _enter();
```

ماذا يفعل؟ حالة `ok`: ندخل.

لماذا بهذه الطريقة؟ `_enter` يجعل الملف نشطًا وينتقل للرئيسية.

الأسطر 72–77

```
72        case PinCheck.wrong:
73          setState(() {
74            _wrong = true;
75            _entry = '';
76            _busy = false;
77          });
```

ماذا يفعل؟ حالة `wrong`: نعلّم الخطأ ونفرّغ الإدخال ونرفع الانشغال.

لماذا بهذه الطريقة؟ النقاط تفرغ وتلوّن بالمرجاني فيرى الطالب أن عليه المحاولة ثانية.

الأسطر 78–85

```
78        case PinCheck.locked:
79          _startCountdown(result.lockedFor ?? const Duration(seconds: 30));
80          setState(() {
81            _wrong = false;
82            _entry = '';
83            _busy = false;
84          });
85      }
```

ماذا يفعل؟ حالة `locked`: نبدأ العدّ التنازلي بالمدة المتبقية ونفرّغ الإدخال.

لماذا بهذه الطريقة؟ 30 ثانية احتياطي إن لم تصل المدة.

السطر 86

```
86    }
```

ماذا يفعل؟ إغلاق `_verify`.

لماذا بهذه الطريقة؟ نهاية الدالة.

الأسطر 88–91

```
88    Future<void> _enter() async {
89      await ref.read(activeProfileIdProvider.notifier).select(widget.profileId);
90      if (mounted) context.go(AppRoutes.home);
91    }
```

ماذا يفعل؟ `_enter`: نجعل الملف نشطًا ثم نفتح الرئيسية.

لماذا بهذه الطريقة؟ `go` فلا يرجع الطالب لشاشة الرمز بزر الرجوع.

الأسطر 93–95

```
93    void _startCountdown(Duration duration) {
94      _ticker?.cancel();
95      _lockedSeconds = (duration.inMilliseconds / 1000).ceil();
```

ماذا يفعل؟ `_startCountdown`: نلغي أي مؤقّت سابق ونحوّل المدة إلى ثوانٍ مقرّبة للأعلى.

لماذا بهذه الطريقة؟ مثلًا 29.4 ثانية تظهر 30.

الأسطر 96–100

```
96      _ticker = Timer.periodic(const Duration(seconds: 1), (timer) {
97        if (!mounted) return;
98        setState(() => _lockedSeconds = (_lockedSeconds - 1).clamp(0, 9999));
99        if (_lockedSeconds == 0) timer.cancel();
100     });
```

ماذا يفعل؟ مؤقّت دوري كل ثانية يخفض العدّاد حتى الصفر (مع `clamp`) ثم يتوقف.

لماذا بهذه الطريقة؟ عند الصفر تُفتح اللوحة تلقائيًا.

السطر 101

```
101   }
```

ماذا يفعل؟ إغلاق الدالة.

لماذا بهذه الطريقة؟ نهاية `_startCountdown` .

الأسطر 103–105

```
103   Future<void> _forgot() async {
104     final profile = ref.read(profileByIdProvider(widget.profileId)).value;
105     if (profile == null) return;
```

ماذا يفعل؟ `_forgot`: نقرأ الملف الحالي ونخرج إن لم يوجد.

لماذا بهذه الطريقة؟ لا ورقة بلا ملف.

الأسطر 106–110

```
106     final changed = await showForgotPinSheet(
107       context,
108       repository: ref.read(profileRepositoryProvider),
109       profile: profile,
110     );
```

ماذا يفعل؟ نفتح ورقة «نسيت الرمز؟» بالمستودع والملف.

لماذا بهذه الطريقة؟ تعيد `true` إن تغيّر الرمز.

الأسطر 111–112

```
111     if (changed && mounted) await _enter();
112   }
```

ماذا يفعل؟ إن تغيّر الرمز ندخل مباشرة.

لماذا بهذه الطريقة؟ أثبت الطالب ملكيته برمز الطالب.

الأسطر 114–116

```
114   @override
115   Widget build(BuildContext context) {
116     final profile = ref.watch(profileByIdProvider(widget.profileId)).value;
```

ماذا يفعل؟ `build` ونقرأ الملف بـ `watch`.

لماذا بهذه الطريقة؟ ليعرض اسمه ولونه.

الأسطر 118–121

```
118     return Scaffold(
119       body: SafeArea(
120         child: CustomScrollView(
121           slivers: [
```

ماذا يفعل؟ `Scaffold` و`SafeArea` و`CustomScrollView`.

لماذا بهذه الطريقة؟ تمرير يتكيّف مع القصير.

الأسطر 122–134

```
122             SliverToBoxAdapter(
123               child: Align(
124                 alignment: AlignmentDirectional.centerStart,
125                 child: Padding(
126                   padding: const EdgeInsetsDirectional.fromSTEB(20, 16, 20, 4),
127                   child: BackTile(
128                     onTap: () => context.canPop()
129                         ? context.pop()
130                         : context.go(AppRoutes.profiles),
131                   ),
132                 ),
133               ),
134             ),
```

ماذا يفعل؟ شريط علوي فيه زر الرجوع فقط في بداية السطر (اليمين).

لماذا بهذه الطريقة؟ الرجوع: `pop` أو ننتقل لشاشة الاختيار.

الأسطر 135–138

```
135             SliverFillRemaining(
136               hasScrollBody: false,
137               child: Padding(
138                 padding: const EdgeInsetsDirectional.fromSTEB(24, 8, 24, 24),
```

ماذا يفعل؟ `SliverFillRemaining` وحشوة 24/8/24/24.

لماذا بهذه الطريقة؟ يسمح لـ `Spacer` بدفع «نسيت الرمز؟» للأسفل.

الأسطر 139–140

```
139                 child: profile == null
140                     ? const SizedBox.shrink()
```

ماذا يفعل؟ إن لم يُوجد الملف نعرض فراغًا.

لماذا بهذه الطريقة؟ ملف محذوف مثلًا.

الأسطر 141–143

```
141                     : Column(
142                         spacing: 14,
143                         children: [
```

ماذا يفعل؟ وإلا `Column` بمسافة 14.

لماذا بهذه الطريقة؟ ترتيب العناصر من التصميم.

الأسطر 144–153

```
144                           Container(
145                             width: 112,
146                             height: 112,
147                             decoration: BoxDecoration(
148                               color: profile.color.avatarBackground,
149                               shape: BoxShape.circle,
150                             ),
151                             alignment: Alignment.center,
152                             child: Mascot(size: 84, color: profile.color),
153                           ),
```

ماذا يفعل؟ أفاتار 112 بخلفية لون الطالب وفيه الشخصية 84.

لماذا بهذه الطريقة؟ يذكّر الطالب أي ملف يفتح.

الأسطر 154–158

```
154                           Text(
155                             ArStrings.pinHello(profile.name),
156                             textAlign: TextAlign.center,
157                             style: AppTextStyles.title26,
158                           ),
```

ماذا يفعل؟ «مرحبًا {الاسم}» بنمط `title26`.

لماذا بهذه الطريقة؟ ترحيب شخصي.

الأسطر 159–165

```
159                     Text(
160                       ArStrings.pinEnterTitle,
161                       textAlign: TextAlign.center,
162                       style: AppTextStyles.body15.copyWith(
163                         color: AppColors.soft,
164                       ),
165                     ),
```

ماذا يفعل؟ «أدخل رمزك السري» باهتًّا.

لماذا بهذه الطريقة؟ `body15` بلون `soft`.

الأسطر 166–170

```
166                     PinDots(
167                       length: PinHasher.pinLength,
168                       filled: _entry.length,
169                       error: _wrong,
170                     ),
```

ماذا يفعل؟ `PinDots` بعدد الأرقام وحالة الخطأ.

لماذا بهذه الطريقة؟ مؤشر بصري.

الأسطر 171–174

```
171                     _StatusMessage(
172                       wrong: _wrong,
173                       lockedSeconds: _lockedSeconds,
174                     ),
```

ماذا يفعل؟ `_StatusMessage`: رسالة الخطأ أو القفل.

لماذا بهذه الطريقة؟ مساحة ثابتة تحفظ التخطيط.

الأسطر 175–179

```
175                     PinPad(
176                       onDigit: _onDigit,
177                       onDelete: _onDelete,
178                       enabled: !_isLocked && !_busy,
179                     ),
```

ماذا يفعل؟ `PinPad` تُعطَّل أثناء القفل أو التحقق.

لماذا بهذه الطريقة؟ `enabled: !_isLocked && !_busy`.

السطر 180

```
180                     const Spacer(),
```

ماذا يفعل؟ `Spacer`.

لماذا بهذه الطريقة؟ يدفع الرابط للأسفل.

الأسطر 181–190

```
                          TextButton(
                            onPressed: _forgot,
                            child: Text(
                              ArStrings.pinForgot,
                              textAlign: TextAlign.center,
                              style: AppTextStyles.label14.copyWith(
                                color: AppColors.teal,
                              ),
                            ),
                          ),
```

ماذا يفعل؟ زر «نسيت الرمز؟» بنص teal.

لماذا بهذه الطريقة؟ نص طويل يلتفّ بمحاذاة المنتصف.

الأسطر 191–200

```
                        ],
                      ),
                ),
              ),
            ],
          ),
        ),
      );
    }
}
```

ماذا يفعل؟ إغلاق الأقواس والصنف.

لماذا بهذه الطريقة؟ نهاية `PinEntryScreen`.

الأسطر 202–206

```
class _StatusMessage extends StatelessWidget {
  const _StatusMessage({required this.wrong, required this.lockedSeconds});

  final bool wrong;
  final int lockedSeconds;
```

ماذا يفعل؟ ودجت `_StatusMessage` وحقلاه.

لماذا بهذه الطريقة؟ الرسالة تحت النقاط.

الأسطر 208–212

```
  @override
  Widget build(BuildContext context) {
    final text = lockedSeconds > 0
        ? ArStrings.pinLocked(lockedSeconds)
        : (wrong ? ArStrings.pinWrong : null);
```

ماذا يفعل؟ `build`: النص: رسالة القفل إن كانت هناك ثوانٍ، وإلا رسالة الخطأ إن كان `wrong`، وإلا لا شيء.

لماذا بهذه الطريقة؟ القفل له أولوية على الخطأ.

الأسطر 214–216

```
    return SizedBox(
      height: 24,
      child: text == null
```

ماذا يفعل؟ `SizedBox` ارتفاعه 24 ومحتواه فارغ إن لم يوجد نص.

لماذا بهذه الطريقة؟ مساحة ثابتة فلا تقفز الأزرار.

الأسطر 217–225

```
          ? null
          : Text(
              text,
              textAlign: TextAlign.center,
              style: AppTextStyles.label14.copyWith(color: AppColors.coral),
            ),
    );
  }
}
```

ماذا يفعل؟ نص بلون مرجاني ومحاذاة منتصف ثم إغلاق الأقواس.

لماذا بهذه الطريقة؟ اللون الأحمر/المرجاني يدل على خطأ.

lib/features/profiles/forgot_pin_sheet.dart — «نسيت الرمز؟»

lib/features/profiles/forgot_pin_sheet.dart

الأسطر 1–2

```
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
```

ماذا يفعل؟ استيراد Material والخدمات.

لماذا بهذه الطريقة؟ `services` للمرشّحات.

الأسطر 4–11

```
import '../../core/db/app_database.dart';
import '../../core/strings/ar.dart';
import '../../core/theme/app_colors.dart';
import '../../core/theme/app_text_styles.dart';
import '../../core/utils/student_code.dart';
import '../../core/widgets/primary_button.dart';
import 'pin_setup_sheet.dart';
import 'profile_repository.dart';
```

ماذا يفعل؟ استيراد القاعدة (لنوع `Profile`) والنصوص والألوان والأنماط ورمز الطالب والزر وورقة اختيار الرمز والمستودع.

لماذا بهذه الطريقة؟ نعيد استعمال ورقة اختيار الرمز نفسها.

الأسطر 13–14

```
/// «نسيت الرمز؟»: يطلب رمز الطالب، فإن صحّ يتيح تعيين رمز جديد أو إلغاءه.
/// يعيد true (فيدخل الطالب بعدها مباشرة) إن تغيّر الرمز السري.
```

ماذا يفعل؟ تعليق الدالة العامة.

لماذا بهذه الطريقة؟ تعيد `true` إن تغيّر الرمز السري.

الأسطر 15–19

```
Future<bool> showForgotPinSheet(
  BuildContext context, {
  required ProfileRepository repository,
  required Profile profile,
}) async {
```

ماذا يفعل؟ توقيع `showForgotPinSheet` بمعاملات مسمّاة: المستودع والملف.

لماذا بهذه الطريقة؟ تمرير المستودع بدل قراءته هنا يبقي الورقة بلا اعتماد على Riverpod فتُختبر بسهولة.

الأسطر 20–28

```
  final changed = await showModalBottomSheet<bool>(
    context: context,
    isScrollControlled: true,
    backgroundColor: AppColors.ground,
    shape: const RoundedRectangleBorder(
      borderRadius: BorderRadius.vertical(top: Radius.circular(28)),
    ),
    builder: (_) => _ForgotPinSheet(repository: repository, profile: profile),
  );
```

ماذا يفعل؟ نفتح ورقة سفلية بخلفية `ground` وزوايا علوية 28.

لماذا بهذه الطريقة؟ `isScrollControlled` لترتفع فوق لوحة المفاتيح.

الأسطر 29–30

```
  return changed ?? false;
}
```

ماذا يفعل؟ نعيد النتيجة أو `false` إن أُغلقت.

لماذا بهذه الطريقة؟ الإغلاق بلا اختيار = لم يتغيّر شيء.

الأسطر 32–40

```
class _ForgotPinSheet extends StatefulWidget {
  const _ForgotPinSheet({required this.repository, required this.profile});

  final ProfileRepository repository;
  final Profile profile;

  @override
  State<_ForgotPinSheet> createState() => _ForgotPinSheetState();
}
```

ماذا يفعل؟ ودجت `_ForgotPinSheet` بمستودع وملف و`createState`.

لماذا بهذه الطريقة؟ حالة محلية.

الأسطر 42–46

```
class _ForgotPinSheetState extends State<_ForgotPinSheet> {
  final _controller = TextEditingController();
  bool _verified = false;
  bool _wrongCode = false;
  bool _busy = false;
```

ماذا يفعل؟ الحالة: الـ controller وعلامات: تم التحقق، رمز خاطئ، انشغال.

لماذا بهذه الطريقة؟ تحدد أي واجهة تظهر.

الأسطر 48–52

```
  @override
  void dispose() {
    _controller.dispose();
    super.dispose();
  }
```

ماذا يفعل؟ `dispose`.

لماذا بهذه الطريقة؟ نتخلّص من الـ controller.

الأسطر 54–59

```
  Future<void> _verify() async {
    setState(() => _busy = true);
    final ok = await widget.repository.matchesStudentCode(
      widget.profile.id,
      _controller.text,
    );
```

ماذا يفعل؟ `_verify`: نضع الانشغال ونسأل المستودع هل الرمز يطابق.

لماذا بهذه الطريقة؟ `matchesStudentCode` تتحقق فقط ولا تغيّر شيئًا.

الأسطر 60–65

```
60      if (!mounted) return;
61      setState(() {
62        _busy = false;
63        _verified = ok;
64        _wrongCode = !ok;
65      });
```

ماذا يفعل؟ نحدّث الحالة: الانشغال انتهى، والتحقق حسب النتيجة، والخطأ عكسها.

لماذا بهذه الطريقة؟ إن صح الرمز تتبدّل الواجهة إلى الخيارين.

السطر 66

```
66    }
```

ماذا يفعل؟ إغلاق الدالة.

لماذا بهذه الطريقة؟ نهاية `_verify`.

الأسطر 68–70

```
68    Future<void> _setNewPin() async {
69      final pin = await showPinSetupSheet(context);
70      if (pin == null || !mounted) return;
```

ماذا يفعل؟ `_setNewPin`: نفتح ورقة اختيار رمز جديد ونخرج إن أُغلقت.

لماذا بهذه الطريقة؟ لا نغيّر شيئًا إن لم يختر الطالب.

الأسطر 71–77

```
71      await widget.repository.resetPinWithStudentCode(
72        widget.profile.id,
73        _controller.text,
74        newPin: pin,
75      );
76      if (mounted) Navigator.of(context).pop(true);
77    }
```

ماذا يفعل؟ نعيّن الرمز الجديد بالمستودع ثم نغلق الورقة بـ `true`.

لماذا بهذه الطريقة؟ المستودع يتحقق مجددًا من رمز الطالب قبل التغيير.

الأسطر 79–85

```
79    Future<void> _removePin() async {
80      await widget.repository.resetPinWithStudentCode(
81        widget.profile.id,
82        _controller.text,
83      );
84      if (mounted) Navigator.of(context).pop(true);
85    }
```

ماذا يفعل؟ `_removePin`: نلغي الرمز السري (بلا رمز جديد) ثم نغلق بـ `true`.

لماذا بهذه الطريقة؟ خيار «إلغاء الرمز السري».

الأسطر 87–91

```
87    @override
88    Widget build(BuildContext context) {
89      return SafeArea(
90        child: SizedBox(
91          width: double.infinity,
```

ماذا يفعل؟ `build` و`SafeArea` و`SizedBox` بعرض كامل.

لماذا بهذه الطريقة؟ العرض الكامل مطلوب كي لا تضيق الورقة.

الأسطر 92–98

```
92          child: Padding(
93            padding: EdgeInsetsDirectional.fromSTEB(
94              24,
95              24,
96              24,
97              16 + MediaQuery.viewInsetsOf(context).bottom,
98            ),
```

ماذا يفعل؟ حشوة 24/24/24/16 مع إضافة ارتفاع لوحة المفاتيح من `viewInsets`.

لماذا بهذه الطريقة؟ ترفع الورقة فوق لوحة المفاتيح.

الأسطر 99–103

```
99            child: Column(
100             mainAxisSize: MainAxisSize.min,
101             crossAxisAlignment: CrossAxisAlignment.stretch,
102             spacing: 14,
103             children: [
```

ماذا يفعل؟ `Column` بحجم أدنى ومسافة 14.

لماذا بهذه الطريقة؟ تأخذ الورقة ما تحتاجه فقط.

الأسطر 104–108

```
104               Text(
105                 ArStrings.forgotTitle,
106                 textAlign: TextAlign.center,
107                 style: AppTextStyles.section18,
108               ),
```

ماذا يفعل؟ عنوان «استعادة الوصول».

لماذا بهذه الطريقة؟ نمط `section18`.

السطر 109

```
109               if (!_verified) ..._codeEntry() else ..._choices(),
```

ماذا يفعل؟ نعرض واجهة إدخال الرمز إن لم يُتحقَّق بعد، وإلا الخيارين.

لماذا بهذه الطريقة؟ `..._codeEntry()` و`..._choices()` تفرد القائمتين داخل الأبناء.

الأسطر 110–115

```
            ],
          ),
        ),
      ),
    );
  }
```

ماذا يفعل؟ إغلاق الأقواس.

لماذا بهذه الطريقة؟ نهاية `build`.

الأسطر 117–118

```
  List<Widget> _codeEntry() {
    const maxLength = 3 + StudentCode.randomLength;
```

ماذا يفعل؟ `_codeEntry` وطول رمز الطالب = البادئة (3) + الجزء العشوائي.

لماذا بهذه الطريقة؟ الطول من `StudentCode` فلو تغيّر لا ينكسر.

الأسطر 119–123

```
    final borderColor = _wrongCode ? AppColors.coral : AppColors.line;
    final border = OutlineInputBorder(
      borderRadius: BorderRadius.circular(14),
      borderSide: BorderSide(color: borderColor, width: 2),
    );
```

ماذا يفعل؟ لون الحد (مرجاني للخطأ) وبناء الحد المشترك.

لماذا بهذه الطريقة؟ نفس نمط حقل رمز الصف.

الأسطر 125–126

```
    return [
      Text(ArStrings.forgotCodeLabel, style: AppTextStyles.body16Strong),
```

ماذا يفعل؟ قائمة تبدأ بتسمية «أدخل رمز الطالب».

لماذا بهذه الطريقة؟ نص فوق الحقل.

الأسطر 127–129

```
      SizedBox(
        height: 52,
        child: TextField(
```

ماذا يفعل؟ `SizedBox` ارتفاعه 52 و`TextField`.

لماذا بهذه الطريقة؟ ارتفاع Figma.

الأسطر 130–135

```
        controller: _controller,
        autofocus: true,
        textDirection: TextDirection.ltr,
        textCapitalization: TextCapitalization.characters,
        cursorColor: AppColors.teal,
        style: AppTextStyles.input17,
```

ماذا يفعل؟ الـ controller وتركيز تلقائي واتجاه LTR وأحرف كبيرة ولون مؤشر ونمط.

لماذا بهذه الطريقة؟ الرمز لاتيني فاتجاهه LTR حتى في واجهة RTL.

الأسطر 136–137

```
        // نعيد البناء مع كل حرف ليتفعّل زر «تحقّق» عند وجود نص.
        onChanged: (_) => setState(() => _wrongCode = false),
```

ماذا يفعل؟ `onChanged`: يمسح علامة الخطأ ويعيد البناء مع كل حرف.

لماذا بهذه الطريقة؟ بدونه لا يتفعّل زر «تحقّق» أبدًا لأن شرط التعطيل يُحسب عند البناء؛ وقد اكتشفنا هذا الخطأ باختبار.

الأسطر 138–144

```
        inputFormatters: [
          FilteringTextInputFormatter.allow(RegExp('[A-Za-z0-9-]')),
          LengthLimitingTextInputFormatter(maxLength),
          TextInputFormatter.withFunction(
            (old, value) => value.copyWith(text: value.text.toUpperCase()),
          ),
        ],
```

ماذا يفعل؟ مرشّحات: حروف وأرقام وشرطة فقط، وطول أقصى، وأحرف كبيرة.

لماذا بهذه الطريقة؟ تمنع الإدخال غير الصالح وتطابق شكل `RZ-XXXX`.

الأسطر 145–160

```
        decoration: InputDecoration(
          hintText: ArStrings.forgotCodeHint,
          hintStyle: AppTextStyles.input17.copyWith(
            color: AppColors.placeholder,
          ),
          filled: true,
          fillColor: AppColors.card,
          border: border,
          enabledBorder: border,
          focusedBorder: border.copyWith(
            borderSide: BorderSide(
              color: _wrongCode ? AppColors.coral : AppColors.teal,
              width: 2,
            ),
          ),
        ),
```

ماذا يفعل؟ زخرفة الحقل: نص إرشادي `RZ-XXXX` وخلفية بيضاء والحدود (تنقلب للمرجاني عند الخطأ).

لماذا بهذه الطريقة؟ مظهر متسق مع بقية الحقول.

الأسطر 161–162

```
        ),
      ),
```

ماذا يفعل؟ إغلاق الحقل.

لماذا بهذه الطريقة؟ نهاية `TextField`.

الأسطر 163–167

```
      if (_wrongCode)
        Text(
          ArStrings.forgotCodeWrong,
          style: AppTextStyles.label14.copyWith(color: AppColors.coral),
        ),
```

ماذا يفعل؟ إن كان الرمز خاطئًا نعرض «رمز الطالب غير صحيح.» بلون مرجاني.

لماذا بهذه الطريقة؟ رسالة واضحة.

الأسطر 168–171

```
      PrimaryButton(
        label: ArStrings.forgotVerify,
        onPressed: _busy || _controller.text.trim().isEmpty ? null : _verify,
      ),
```

ماذا يفعل؟ زر «تحقّق»: يُعطَّل أثناء الانشغال أو عند الحقل الفارغ.

لماذا بهذه الطريقة؟ `null` يعطّل الزر.

الأسطر 172–173

```
    ];
  }
```

ماذا يفعل؟ إغلاق القائمة والدالة.

لماذا بهذه الطريقة؟ نهاية `_codeEntry`.

الأسطر 175–181

```
  List<Widget> _choices() {
    return [
      Text(
        ArStrings.forgotChoose,
        textAlign: TextAlign.center,
        style: AppTextStyles.body15,
      ),
```

ماذا يفعل؟ `_choices`: نص «تم التحقق. ماذا تريد أن تفعل؟».

لماذا بهذه الطريقة؟ يظهر بعد نجاح التحقق.

السطر 182

```
182        PrimaryButton(label: ArStrings.forgotSetNew, onPressed: _setNewPin),
```

ماذا يفعل؟ زر «تعيين رمز سري جديد».

لماذا بهذه الطريقة؟ يفتح ورقة اختيار الرمز.

الأسطر 183–188

```
183        PrimaryButton(
184          label: ArStrings.forgotRemove,
185          onPressed: _removePin,
186          backgroundColor: AppColors.mute,
187          foregroundColor: AppColors.ink,
188        ),
```

ماذا يفعل؟ زر «إلغاء الرمز السري» بخلفية `mute` ونص `ink`.

لماذا بهذه الطريقة؟ خيار ثانوي بمظهر أهدأ.

الأسطر 189–191

```
189      ];
190    }
191  }
```

ماذا يفعل؟ إغلاق القائمة والدالة والصنف.

لماذا بهذه الطريقة؟ نهاية الملف.